Customer Credit Control

Technical guide (technical overview)

Technical name	`odt_customer_credit_control`
Module version	`19.0.1.0.0`
Odoo version	19.0
Document type	Technical guide (technical overview)
Language	English
Generated on	2026-10-15

Contents

Overview 2
- What the module does 2
- Design 2

Dependencies 4
- Odoo modules 4
- External dependencies 4

Prerequisites 5
- Installation order 5
- Required configuration 5
- Required data 5
- Access rights 5

Quality 7
- Tests 7
- Licence and authorship 7
- Compliance with the OdooTec module standard 7

Open points 8
- Unfinished work 8
- Translations 9
- Reports 9
- Documentation gaps 9

Overview

What the module does

The module enforces a company credit policy on sales. Each company has a monthly credit ceiling and each customer a monthly cap expressed as a share of that ceiling. Confirming a credit order that would push the customer past its cap, or the company past its ceiling, leaves the order unconfirmed and waiting for a Finance Manager's approval. Independently, every customer carries a credit state (Normal, Watch List, Credit Hold, Absolute Block) that a nightly scheduled action derives from overdue invoices and that Finance can set by hand. Held customers cannot confirm orders or receive deliveries; blocked customers additionally cannot have quotations or customer invoices posted.

Design

Models

Model	New / inherited	Role
`credit.exposure`	New, SQL view (`_auto = False`)	One row per commercial partner, company and calendar month: credit sales with VAT in company currency, the ceiling, the share and the cap. Single source of truth for every total (`models/credit_exposure.py:83`).
`credit.decision`	New, transient	Dialog that captures a decision and its mandatory reason, then calls the matching action on `res.partner` or `sale.order` (`wizard/credit_decision.py:80`).
`res.partner`	Inherited	`credit_state` , `credit_cap_pct` , `is_credit_flagged` , `credit_state_reason` , `hold_cleared_invoice_ids` ; Finance actions; the two cron methods.
`res.company`	Inherited	`credit_ceiling` (0 = no ceiling) and `hold_days` (default 60).
`sale.order`	Inherited	`credit_approval` state, approver, date and reason; computed `credit_share_pct` ; override of `action_confirm` ; partial index `_credit_company_date_idx` .
`account.payment.term`	Inherited	`advance_pct` ; a term at 100 makes its orders non-credit sales.
`product.template`	Inherited	`is_special` and `is_credit_excluded` ; lines of products with both flags are left out of credit totals.
`stock.picking`	Inherited	Override of `button_validate` for outgoing pickings.
`account.move`	Inherited	Override of `action_post` for `out_invoice` .

Key flows

- **Confirmation check.** `sale.order.action_confirm` (`models/sale_order.py:144`) first raises `UserError` for customers on hold or block, then filters the orders that need approval (`_needs_credit_approval`, line 263). Those are written `credit_approval = 'pending'`, get a chatter message and are left out of the `super()` call; the method returns a sticky warning notification instead of the standard result. The others confirm as standard.
- **Limit computation.** `_needs_credit_approval` flushes, invalidates the `credit.exposure` cache (its ids are positional `ROW_NUMBER()`), reads the company's rows of the current month with `sudo()`, adds the order's own credit amount (`_credit_amount`, line 245) and compares with `currency.compare_amounts`. Exactly on the limit confirms. A ceiling of 0 disables both checks; a cap of 0 disables the customer check only.
- **Approval.** `action_credit_approve` (line 198) checks `account.group_account_manager` and a non-empty reason, writes approver, date and reason under `sudo()`, then calls `action_confirm` again; `approved` orders skip the limit check.
- **Credit states.** All Finance actions on `res.partner` (lines 121 to 240) check rights in `_check_credit_right`, require a reason, and write with `sudo()` because, as the code comments say, Finance users cannot edit contacts in Odoo 19. `write()` is overridden (line 93) to refuse direct edits of the credit fields by users outside the Finance groups.
- **Nightly check.** `_cron_credit_check` (line 244, cron `ir_cron_credit_check`, daily at 00:30) runs as superuser, groups overdue `out_invoice` by commercial partner, never touches customers on hold or block, sets Hold when an invoice is more than `hold_days` overdue and not in `hold_cleared_invoice_ids`, Watch when anything is overdue or the Finance flag is set, otherwise Normal. Paid invoices are removed from the covered list.
- **Weekly email.** `_cron_credit_watch_report` (line 306, cron `ir_cron_credit_watch_report`, weekly) creates one `mail.mail` per company to the Finance Managers of that company who have an email address. A dry run on the documentation database on 2026-10-15 (rolled back) produced the expected subject and table.
- **Blocks elsewhere.** `stock.picking.button_validate` refuses outgoing pickings of held or blocked customers; `account.move.action_post` refuses customer invoices of blocked customers; the constraint `_check_partner_credit_block` (`models/sale_order.py:123`) refuses saving a draft or sent quotation for a blocked customer.

Notable decisions and trade-offs

- **SQL view instead of stored totals.** Totals cannot drift from the orders, at the cost of a query at each confirmation. The partial index on `sale_order (company_id, date_order) WHERE state = 'sale'` supports it.
- **Month of `date_order` in UTC.** The view truncates `so.date_order` (a UTC timestamp) and the check uses `fields.Datetime.now()`; an order confirmed shortly after local midnight on the first day of a month can fall in the previous month. The README states this limitation.
- **Rights checked in code, writes under `sudo()`.** This lets Finance act without contact or sales write rights, but makes the code the only guard: the ACL of `credit.decision` alone does not protect anything.
- No `post_init_hook` or `uninstall_hook` is declared. The crons are in a `noupdate="1"` block.

Dependencies

Odoo modules

Module	Why it is needed
`sale_management`	Sales orders, the confirmation override, the order form and the Sales reporting menu that hosts **Credit Exposure**.
`stock`	`stock.picking.button_validate` override for deliveries.
`account`	Invoices (overdue detection, posting block), payment terms (`advance_pct`), the Finance groups `account.group_account_invoice` and `account.group_account_manager`.

The view `views/sale_order_views.xml` targets a `div.alert-warning[@role='alert']` of `sale.view_order_form`, and `models/sale_order.py:257` reads `sale.order.line.is_downpayment`; both come from `sale`, pulled in by `sale_management`.

External dependencies

None declared. The code imports only `odoo` and `markupsafe` (shipped with Odoo). The tests import `freezegun` (`tests/common.py:13`), which is a standard Odoo test requirement and is not declared in the manifest.

Prerequisites

Installation order

Install after `sale_management`, `stock` and `account` (resolved automatically by the dependency list). The sibling module `odt_quotation_payment_terms` builds on this one (README, "Architectural Position") and should be installed after it.

Required configuration

- **Company:** set **Monthly Credit Ceiling** and **Credit Hold After (Days Overdue)** on the company form, tab **Credit Control** (visible to `account.group_account_manager` only). With a ceiling of 0, no order ever waits for approval.
- **Customers:** set **Monthly Credit Cap (%)** on the commercial partner. 0 means no customer cap.
- **Payment terms:** set **Advance (%)**; 100 means the term is not a credit sale. Every existing term starts at 0, which makes every order a credit sale.
- **Products:** mark **Special Item** and **Excluded from Credit Totals** where the policy needs it.
- **Scheduled actions:** Credit Control: nightly credit check and Credit Control: weekly watch-list email are active on install.
- **Outgoing mail:** the weekly email is a `mail.mail` record; it needs an outgoing mail server, and Finance Managers need an email address on their contact.

Required data

No data is required beyond the configuration above. The module ships no sequences, journals or demo data.

Access rights

Group	What it can do in this module
`sales_team.group_sale_salesman`	Read `credit.exposure` (record rule: own companies), see the **Credit Exposure** menu, see share and cap on quotations.
`account.group_account_invoice` (Invoicing)	Put a customer on the watch list and remove the Finance flag through `credit.decision`.
`account.group_account_manager` (Finance Manager)	Everything above, plus hold, lift hold, block, lift block, approve orders over a limit, edit the company **Credit Control** tab, see the cap and the covered invoices on the customer.

Record rule `credit_exposure_rule_company` limits exposure rows to the user's companies (`security/security.xml`). `credit.decision` has no unlink right for anyone (normal for a transient model).

> **Note:** a probe on the documentation database (2026-10-15, rolled back) showed that a user with only **Accounting / Administrator** gets an `AccessError` when saving **Monthly Credit Cap (%)** on a customer ("not allowed to modify 'Contact'"), and cannot save the company ceiling either (needs **Administration / Access Rights**). The Finance actions avoid this through `sudo()`, but the cap and the ceiling are plain form fields. Whoever maintains them also needs contact edit rights (for example **Contacts / Creation** or **Sales / Administrator**) and settings rights respectively. The README (step 3 of "Installation & Configuration") does not say so.

Quality

Tests

32 test methods in 3 files, all imported by `tests/__init__.py`, tagged `post_install`, `-at_install` and `odt_customer_credit_control`:

File	Methods	Covers
`tests/test_credit_ceiling.py`	11	Cap and ceiling checks, boundaries, approval, advance terms, excluded special lines, new month, two companies, exposure share, record rule.
`tests/test_credit_states.py`	12	Watch list, hold after more than 60 days, lifted hold not reset, manual flag, refusals, weekly report order, Finance Manager right.
`tests/test_overrides.py`	9	Positive, negative and standard-preserved cases for confirmation, delivery and invoice posting.

Execution: the full suite was executed by the delivery team on 2026-10-15 on a fresh database (the three RVK modules installed with `--test-enable`): 74 tests, 0 failed, 0 errors, of which these 32 belong to this module. The documentation run itself counted the tests and did not execute them.

No test covers the **Over Cap** search filter or the form refresh after confirmation (see Open points).

Licence and authorship

Manifest licence `Other proprietary`; author OdooTec; website https://www.odootec.com; no maintainer declared. A `LICENSE` file is present (OdooTec Development Licence ODTL-1, version 1.4), and the README states the same licence.

Compliance with the OdooTec module standard

Compliance with the OdooTec module standard was not checked: the standard reference file is empty.

Open points

Unfinished work

No TODO, FIXME or XXX comments, stub methods, missing data files or unimported Python files were found. Findings from reading the code and running the screens:

Finding	Evidence	Effect
The **Over Cap** filter does not filter customers over their cap. Its domain is `[('cap_pct', '>', 0)]`, so it lists every customer that has a cap.	`views/credit_exposure_views.xml:54`; on the demo database the filter returned a customer at 1.15 % of a 20 % cap.	Misleading report filter. A correct filter needs a stored or SQL comparison of `share_pct` and `cap_pct` (e.g. an `over_cap` boolean column in the view).
After **Confirm** on an order that must wait, the form is not reloaded. The warning notification appears, but the red banner, the **Approve Credit** button and the chatter message only show after a page reload.	Screenshots taken on 2026-10-15; `models/sale_order.py:186` returns a `display_notification` client action without a follow-up reload.	Users may think nothing happened. Returning the notification with `params.next = {'type': 'ir.actions.client', 'tag': 'soft_reload'}` would refresh the form.
The **Advance (%)** field has no visible label on the payment term form: it is inserted `position="before"` the `early_discount` field, which sits inside the `div` that follows the `<label for="early_discount"/>`, so its value renders on the **Early Discount** line.	`views/account_payment_term_views.xml:8`; `account/views/account_payment_term_views.xml:46-48`; screenshot of 2026-10-15.	Users cannot identify the field. Anchoring the field before the `label` (e.g. `//label[@for='early_discount']` `position="before"`) would give it its own row and label.
The covered-invoices list on the customer's **Credit Control** tab is shown without a label.	`views/res_partner_views.xml:50` (field outside a group, no `string` shown).	Cosmetic.
Cap and ceiling cannot be maintained by a user who has only the Finance Manager group.	Access probe described under Prerequisites.	Configuration gap; document the extra rights or move the fields behind a `sudo()` action.

Finding	Evidence	Effect
Known limitations stated in the README: returned instrument deposits (REQ-026) do not yet trigger the watch list; month boundaries in UTC.	`README.md` , "Known Limitations".	Not verified further.

Translations

`i18n/ar.po` has 125 entries; the static extraction found 0 missing, 0 untranslated and 0 fuzzy strings, and the `.pot` holds every extracted source string. Extraction is static, so it is an approximation of Odoo's own export.

Reports

The module defines no QWeb report. The weekly email is an HTML table built in Python (`models/res_partner.py:425`).

Documentation gaps

- Tests were not run by the documentation run; the cited results come from the delivery team.
- Multi-company behaviour (separate ceilings per company) was not exercised on screen: the documentation database has one company. It is covered by `test_req022_t9_companies_apart` and `test_rule_exposure_company` .
- The weekly email was produced in a rolled-back dry run; actual sending through a mail server was not checked.
- Performance of the SQL view on large order volumes was not measured.